

CONTAINERISATION

Software Engineering in the Real World

CMP9134 Software Engineering — Week 7 — 20 March 2026

Prof. Marc Hanheide

Director, Lincoln Centre for Autonomous Systems (L-CAS) | CTO, JABAS.ai

Start the presentation to see live content. For screen share software, share the entire screen. Get help at pollev.com/app

Have you used Docker or containers before?



Never

0%

Once or Twice

0%

Whenever forced to

0%

I have used it regularly

0%

I'm a Docker/Container Guru

0%

Have you ever said or heard the phrase "But it works on my computer?"



Never heard nor said

0%

Have heard it but never said it myself

0%

Have said it but never heard anyone else saying it

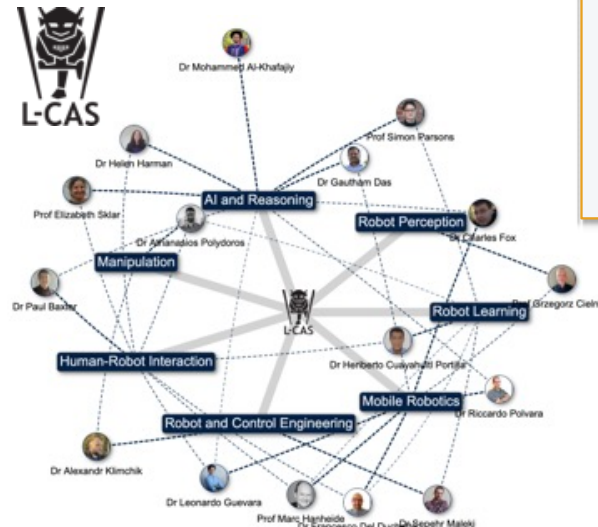
0%

Hear it all the time, and may have said it myself.

0%

About Your Guest Lecturer

- Professor of Intelligent Robotics & Interactive Systems, University of Lincoln
- Director - Lincoln Centre for Autonomous Systems (L-CAS)
- CTO - JABAS.ai (Startup in robotics navigation)
- 20+ years "shipping" software that runs on physical robots (and "in the cloud")



Why I care about containers:

- Our robots run in the field — not in my office
- Team of 20+ researchers, all different machines
- Software must behave identically on lab server, Raspberry Pi, and cloud
- A deployment bug on a physical robot is a safety issue

Learning Objectives

By the end of today you will be able to...

- Explain what containerisation is and why it matters for software engineering
- Describe the key Docker concepts: images, containers, Dockerfiles, registries
- Use Docker Compose to orchestrate multi-container applications
- Configure a devcontainer for reproducible development environments
- Connect containerisation to CI/CD pipelines and your assessment



Assessment 1 Map

- Section 4 — Containerisation 20%
- Artefact (video) — Docker setup
- Section 2 — Architecture diagrams
- Section 3 — GitHub CI/CD evidence
- Robot API IS a container — you need this!

Today's Journey

 The Problem — Dependency Hell

 Docker Fundamentals

 LIVE DEMO — Instant Robot

 Docker Compose

 DevContainers

 Real-World Examples (L-CAS + JABAS)

 Containers for Agentic AI & CI

 Your Assessment

 Q&A

THE PROBLEM

Why does this matter?

"It Works on My Machine"

The Classic Scenario

- Developer writes code on macOS M3
- Pushes to GitHub
- CI server (Ubuntu x86) fails
- Colleague on Windows: 'it crashes immediately'
- Robot (Raspberry Pi ARM): won't even run
- Production server: works, but only sometimes

Root Causes

- Different Python / Node.js versions
- OS-level library differences
- Missing environment variables
- Hardware architecture (x86 vs ARM)
- Global package installations
- "But I installed it last year!"

VMs vs Containers

Both solve environment isolation — but very differently

Virtual Machine

- Full OS inside a VM (GB of disk)
- Slow to start (minutes)
- Strong isolation
- High resource overhead
- Great for: isolated servers

VS

Container

- Shares host OS kernel (MB of disk)
- Starts in seconds (or milliseconds)
- Process-level isolation
- Near-native performance
- Great for: applications & microservices

The Shipping Container Analogy



Malcolm McLean standardised global trade in 1956. Docker did the same for software?!

 Ship	↔	Host OS / Cloud Provider
 Port / Crane	↔	Container Runtime (Docker, containerd)
 Shipping Container	↔	Docker Container Image
 Shipping Manifest	↔	Dockerfile
 Fleet of ships	↔	Docker Compose / Kubernetes
 Any port in the world	↔	Any machine: laptop, server, robot, cloud

DOCKER FUNDAMENTALS

Images · Containers · Dockerfiles · Registries

Docker: The Four Key Concepts



Image

A read-only blueprint. Built from a Dockerfile. Think: class definition.



Container

A running instance of an image. Think: object instance. Isolated process.



Dockerfile

A text recipe: base image + dependencies + config + startup command. Reproducible, “system as code”



Registry

Stores and shares images. Docker Hub (public). GHCR (GitHub). Your own.



One Image → Many Containers. Same blueprint, multiple isolated instances running simultaneously.

Anatomy of a Dockerfile

A step-by-step recipe — each line adds a layer to the image

```
# Start from an official base image
FROM python:3.11-slim

# Set working directory inside the container
WORKDIR /app

# Copy dependency list first (for caching)
COPY requirements.txt .

# Install dependencies
RUN pip install --no-cache-dir -r requirements.txt

# Copy application code
COPY . .

# Expose the port the app listens on
EXPOSE 5000

# Command to run when container starts
CMD ["python", "main.py"]
```

← Base image — Python 3.11, minimal OS

← Isolation: your app lives here, nowhere else

← Copy deps first → Docker caches this layer

← Executed at build time, not runtime, usually you have many of those

← Your actual code copied into the image

← Documents which port to expose, for network access

← Define what shall be executed when container starts

From Dockerfile to Running Container

1 BUILD

```
docker build -t my-app:v1 .
```

Reads Dockerfile (in current directory "."), executes each RUN step, creates image layer cake

2 RUN

```
docker run -p 5000:5000 my-app:v1
```

Creates a container from the image. -p maps host:container ports

3 INSPECT

```
docker ps / docker logs my-app / docker exec -it my-app bash
```

List running containers, view logs, or get a shell inside

4 SHARE

```
docker push ghcr.io/your-org/my-app:v1
```

Push to a registry — anyone can pull and run the exact same image (rarely used, usually done automatically)



LIVE DEMO

Instant Robot — Zero Installation

```
docker history ghcr.io/francescodeiduchetto/cmp9134_2526_robotsim
```

```
docker run -p 5000:5000 ghcr.io/francescodeiduchetto/cmp9134_2526_robotsim:latest
```

Then open: <http://localhost:5000/docs>

- What just happened? Where is Python installed?
- What OS is running inside the container?
- What does the `-p 5000:5000` flag do?
- How would I run 5 copies of this robot simultaneously?

DOCKER COMPOSE

Orchestrating multi-container applications

The Multi-Service Problem

Real applications are never a single container

Your Assessment: Robot Management System



Virtual Robot API

Provided Docker container — the black box



Your Backend

Python / Node / Java — your logic



Your Frontend

Dashboard, 2D map, status display



Database

Persistent mission log / audit trail

Without Compose: 4 separate docker build + docker run commands, manual networking, manual startup order. With Compose: one file, one command.

```
docker-compose up --build
```

docker-compose.yml — The Orchestration File

```
services:

  robot-api:
    image: ghcr.io/francescoduchetto/cmp9134_2526_robotsim:latest
    ports:
      - "5000:5000"
    restart: always

  backend:
    build: ./backend
    ports:
      - "8000:8000"
    depends_on:
      - robot-api
      - database
    environment:
      - ROBOT_URL=http://robot-api:5000

  database:
    image: postgres:15-alpine
    volumes:
      - db_data:/var/lib/postgresql/data

  frontend:
    build: ./frontend
    ports:
      - "3000:3000"
    depends_on:
      - backend

volumes:
  db_data:
```

← Pull the robot from a public registry — no build needed

← Build your backend from local Dockerfile

← Start order: wait for robot-api + db first

← Services communicate by name — 'robot-api' is the hostname

← Official Postgres — version pinned, reproducible

← Named volume — data survives container restarts

DEVCONTAINERS

Your development environment, containerised

DevContainers: The Developer Environment Problem

Dependency hell doesn't stop at runtime — it starts at development time



Without DevContainers

- New team member joins
- Clone repo, follow 47-step README
- Python version mismatch — 2 hours
- Missing system library — 1 hour
- Environment variable not set — 30 min
- 'Works for me, must be your laptop'
- Day 1 productivity: zero



With a DevContainer

- Clone repo
- Open in VS Code → 'Reopen in Container'
- Wait 2 minutes
- Full environment ready: right Python, right libraries, right tools, right config
- Day 1 productivity: immediate

devcontainer.json — The Configuration File

Lives in .devcontainer/ at the root of your project

```
{
  "name": "Robot Management System",

  // Use the docker-compose.yml to define services
  "dockerComposeFile": "../docker-compose.yml",

  // Which service VS Code attaches to
  "service": "backend",

  // Where VS Code opens inside the container
  "workspaceFolder": "/workspace",

  // Extensions installed automatically
  "customizations": {
    "vscode": {
      "extensions": [
        "ms-python.python",
        "ms-python.pylance",
        "ms-azuretools.vscode-docker"
      ]
    }
  },

  // Commands run after container starts
  "postCreateCommand": "pip install -r requirements.txt"
}
```

What this gives you:

- VS Code running inside the container
- Terminal = inside the container
- Debugger = inside the container
- Extensions auto-installed
- Same environment for every developer
- Works locally AND in GitHub Codespaces (and other IDEs!)

 Open standard: containers.dev — works in VS Code, GitHub Codespaces, JetBrains, Gitpod

GitHub Codespaces — DevContainers in the Cloud

Zero installation. Full VS Code. In your browser.

github.com → any repo → green Code button → Codespaces tab → Create codespace



Free for students

60 hours/month free via GitHub Student Developer Pack



Browser-based VS Code

Full editor, terminal, debugger — no installation



DevContainer auto-applied

If your repo has `.devcontainer/`, Codespaces uses it automatically



Instant onboarding

Open a repo, click a button, start coding in 60 seconds



Relevant to your assessment

Add `devcontainer.json` to your project → you could code on your phone ;-)

REAL-WORLD EXAMPLES

How this is actually used in robotics and commercial AI

PhenAlx

Robotic Phenotyping Research Exhibit

University of Lincoln • Lincoln Institute for Agri-Food Technology & Lincoln Centre for Autonomous Systems

How it Works

1. Plant placed in scanning chamber
2. Robot autonomously positions sensors
3. AI analyses plant health & characteristics
4. Visitor received "top Trump" card for their chosen plant

Multi-Modal Sensors

Hyperspectral • LiDAR • RGB

Controlled Environment

Safe robotic operation

Real-Time Analysis

3D plant reconstruction & phenotyping



Real-time Plant Analysis
NDVI: 0.763 ± 0.012
Leaf Area Index: 2.34 m²/m²
Chlorophyll: 47.2 SPAD units
Plant Height: 24.7 cm

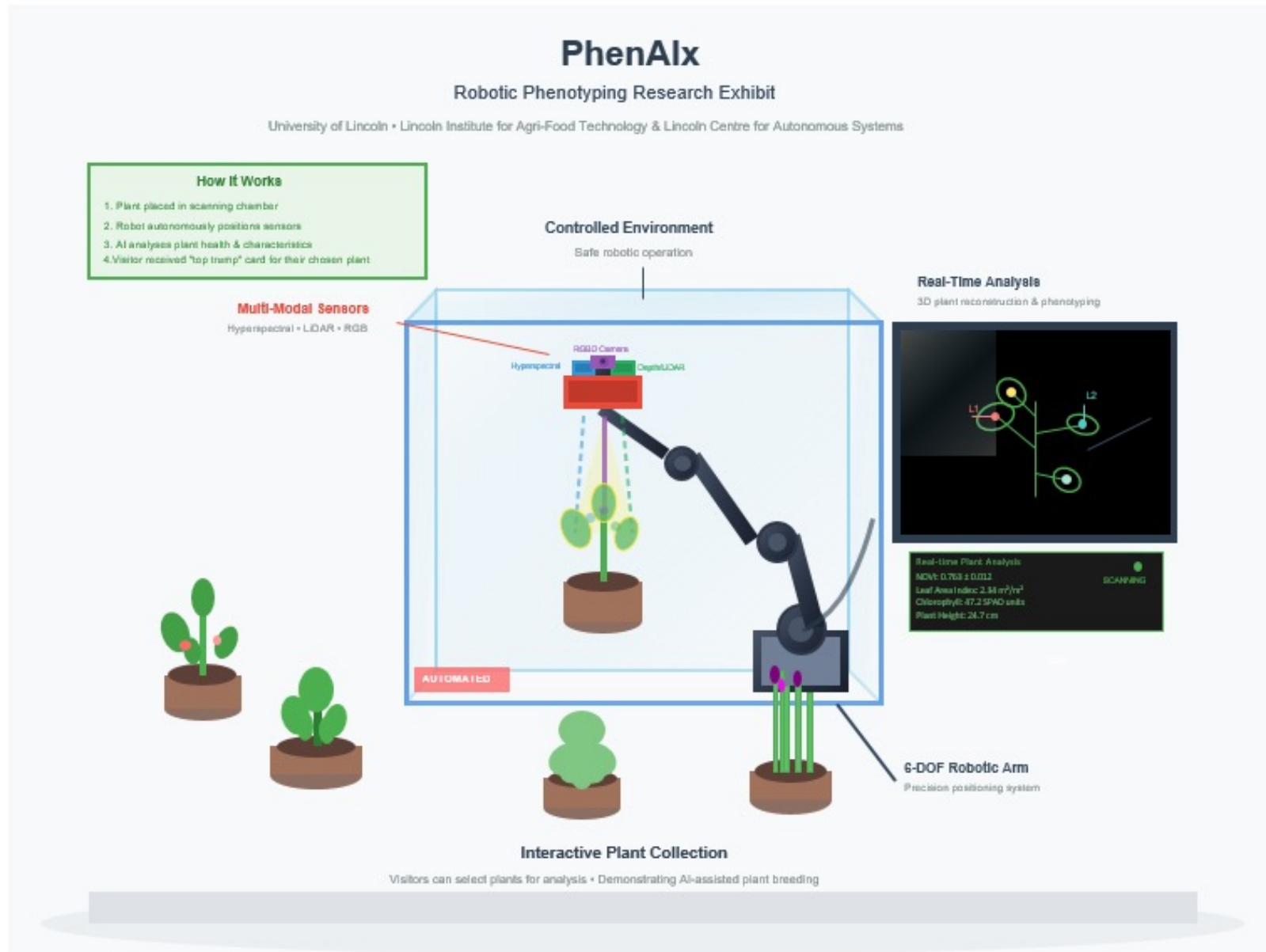
SCANNING

6-DOF Robotic Arm

Precision positioning system

Interactive Plant Collection

Visitors can select plants for analysis • Demonstrating AI-assisted plant breeding



Example 1: PhenAIx (L-CAS)

Standardising development environments for heterogeneous research teams

What is PhenAIx?

- Automated 3D plant scanning system for crop phenotyping research
- UR3e robotic arm + RealSense RGBD camera + Dynamixel servos
- Stack: ROS 2 Jazzy · MoveIt2 · Gazebo · PCL · CUDA 12.6 · Ubuntu 24.04

The heterogeneous team problem

- Researchers join from different schools/departments — all different laptops/computers
- ROS 2 Jazzy basic install alone: 300+ dependencies, specific Ubuntu version
- Simulation (Gazebo) vs. real hardware have different driver requirements
- CUDA version mismatches kill GPU-accelerated point cloud processing
- Short-term projects: student leaves, nobody can reproduce their environment

The solution: two devcontainers

- simulation/ — Gazebo, virtual display, GPU optional
- hardware/ — real UR3e, USB mounts, host network
- Both: ROS2 Jazzy + CUDA 12.6 base image from L-CAS registry
- VNC desktop on :5801 (auto-opens in browser)
- Identical environment: laptop, server, CI, robot
- New researcher: clone → open in VS Code → productive in < 10 min

PhenAIx — Hardware DevContainer Configuration

hardware/devcontainer.json — for real UR3e robot development

```
{
  "name": "L-CAS Jazzy CUDA-OpenGL Devcontainer (Hardware)",
  "build": {
    "dockerfile": "../Dockerfile",
    "args": {
      "BASE_IMAGE": "lcas.lincoln.ac.uk/lcas/ros-docker-images:noble-cuda12.6-jazzy-2"
    },
    "target": "devcontainer",
    "cacheFrom": "lcas.lincoln.ac.uk/devcontainer/lcas/phenaix:latest"
  },

  "forwardPorts": [5801],
  "portsAttributes": {
    "5801": { "label": "desktop", "onAutoForward": "openBrowser" }
  },

  "postStartCommand": "turbovnc.sh /bin/true; .devcontainer/post-create.sh",

  "runArgs": [
    "--privileged",
    "--add-host=ur3e.local:192.168.1.101",
    "--network=host"
  ],

  "mounts": [
    "type=bind,source=/dev/bus/usb,target=/dev/bus/usb",
    "type=bind,source=/dev,target=/dev"
  ],

  "customizations": {
    "vscode": {
      "extensions": ["ms-python.python", "AlessandroSofia.res2-topic-viewer"]
    }
  }
}
```

What each part does:

cacheFrom	Pre-built image from L-CAS registry — fast startup for new team members
target: devcontainer	Multi-stage Dockerfile — separate targets for devcontainer vs production
forwardPorts: [5801]	VNC desktop auto-forwarded and opened in browser
--privileged + mounts	USB device passthrough: RealSense camera, Dynamixel servos
--network=host	Real-time ROS2 comms with physical UR3e arm over LAN
--add-host	Static hostname for robot IP — works regardless of DNS
updateRemoteUserUID	Container user UID matches host — no file permission issues
post-create.sh	rosdep install + colcon build + aliases auto-configured

Example 2: JABAS.ai — Robot Navigation Stack

Complete production system running on a physical robot — all in containers

What is jabas_deploy?

- Production deployment of a full autonomous robot stack — containerised from day one
- github.com/jabasai/jabas_deploy — modular Docker Compose across multiple robots (Hunter, Dogtooth, Dynium platforms)

jabas_navstack	ROS 2 navigation + MongoDB + VNC desktop. The core robot 'brain'
minio	Object storage (MinIO) — persists RoboFlow tasks/playlists
zenoh bridge	ROS2 DDS bridge for cross-network robot comms
xvfb/noVNC	Virtual display + browser-based desktop (simulation mode)
roboflow	Task planner — mission scheduling and execution logic
caddy	Reverse proxy — exposes services via clean URLs
headscale/VPN	Tailscale VPN — secure remote access to the robot
log-uploader	Automatically uploads CAN bus logs to Google Drive

Two profiles: --profile simulation (bridge network, virtual display, dev) vs --profile real (host network, privileged, hardware access, production)

JABAS.ai — Modular Compose Architecture

One command deploys a complete robot — simulation or real hardware

```
# compose.yml — master orchestration file
include:
  - compose.navstack.yml      # ROS2 nav + MongoDB + VNC
  - compose.certificates.yml  # Step CA cert management
  - compose.zenoh.yml         # ROS2 DDS bridge
  - compose.headscale.yml     # Tailscale VPN
  - compose.roboflow.yml      # Task planner + MinIO

services:
  drive-log-uploader:
    profiles: [real]
    image: ghcr.io/marc-hanheide/gdrive-uploader:latest
    restart: unless-stopped
    volumes:
      - ./data/logs/can/:/app/uploads

# ---- compose.navstack.yml (excerpt) ----
jabas_navstack_real:      # PRODUCTION
  profiles: [real]
  extends:
    service: jabas_navstack_base # ← inherits base config
  network_mode: host        # ROS2 needs host networking
  privileged: true
  cap_add: [NET_ADMIN, SYS_PTRACE, SYS_ADMIN]
  volumes:
    - /dev:/dev              # all hardware devices
    - /run/udev:/run/udev:ro

jabas_navstack_simulation: # DEVELOPMENT
  profiles: [simulation, test]
  extends:
    service: jabas_navstack_base
  networks: [jabas_network] # isolated bridge
  ports: ["5801", "7447", "27018", "9999"]
```

Key architectural patterns:

include:

Splits a complex system across multiple files — each subsystem owns its own compose file

extends:

Abstract base service — shared config once, simulation and real variants override only what differs

profiles:

One codebase, two deployment targets — never maintain two separate compose files

Named volumes with env vars

Volume name encodes platform model + version + serial → reproducible, versioned data

Attach to running container

VS Code: 'Attach to Running Container' — full IDE inside the live robot container

CI tests full stack

GitHub Actions spins up simulation profile, runs health checks, posts stats to PR summary

CONTAINERS FOR AI & CI

Agentic coding, automated testing, reproducible pipelines

Containers as AI Agent Sandboxes

The devcontainer is becoming the standard environment for AI coding agents



What AI coding agents need

- A sandboxed environment to run code safely
- Access to the right tools and dependencies
- Ability to build, run, and test — not just write
- Isolation so mistakes don't break the host
- Reproducibility — same result every run

→ A devcontainer satisfies all of these.



How it works in practice

- GitHub Copilot, Claude Code, Cursor, all support devcontainers
- Agent opens your repo → finds .devcontainer/ → runs inside it
- Agent has access to your exact stack: your Python, your ROS2, your tools
- .github/copilot-instructions.md → agent reads project context
- Container boundary = safety boundary: agent can't touch your host



A well-designed devcontainer.json is now dual-purpose: onboarding humans AND onboarding AI agents.

Containers in CI/CD — The Real-World Pattern

Every serious project runs its tests inside containers

The standard CI container workflow:

```
# .github/workflows/test.yml (JABAS pattern)

jobs:
  test:
    runs-on: jabas          # self-hosted robot CI runner
    strategy:
      matrix:
        platform: [platform_a,
                  platform_b]
    steps:
      - uses: actions/checkout@v4

      - name: Login to registry
        run: |
          docker login registry.services.jabas.ai

      - name: Pull/build images
        run: |
          docker compose --profile simulation pull

      - name: Start full stack
        run: |
          docker compose --profile test up -d
          sleep 20 # let services stabilise

      - name: Health checks + report

...
```

What this teaches us:

Test the real thing

No mocks — the full compose stack runs in CI, same as production. If it works in CI, it works on the robot.

Matrix testing

Three robot platforms tested on every push. Containers make this trivial — swap env var, spin up, test, tear down.

Self-hosted runners

Can run on a physical robot lab machine (not in the cloud)
-> the test environment IS the deployment environment.

Isolated + clean

docker compose down -v removes volumes after tests. Every run starts clean. No state leakage between builds.

Your assessment

Add a GitHub Actions workflow that runs docker compose up, hits your API → automatic CI evidence.



Think — Pair — Share

You are engineering a system to control a fleet of 10 mobile robots. Each robot needs to run the same navigation software. A central server collects telemetry and issues mission commands.

? How would you architect this using containers? What services would you define? What would a `docker-compose.yml` look like?



2 minutes — Think individually



3 minutes — Discuss with the person next to you



3 minutes — Share with the class

YOUR ASSESSMENT

How this maps to your marks

Assessment 1: The Container is Everywhere

100% of your module mark — containers are not optional

Section 1 (20%)	Critical Reflection	Why is containerisation the right approach for robot control software? Cite it.
Section 2 (15%)	Project Planning	Architecture diagram MUST show your containers and how they communicate.
Section 3 (15%)	Project Management	GitHub repo: Dockerfile, docker-compose.yml, CI workflow — all version controlled.
Section 4 (20%)	Testing, Building & Containerisation	Docker Compose working. CI builds and pushes image. Documented.
Artefact (20%)	Code + Video	Video: show docker-compose up working. Pick a container function and explain it.
Section 5 (10%)	Ethics	Safety implications of containerised robot software failures.

What 'Distinction' Looks Like — Section 4

Testing, Building & Containerisation (20%)

PASS	MERIT	DISTINCTION
• Basic testing, limited coverage • No containerisation strategy • No automated building	• Automated testing with good coverage • Appropriate automated building • Containerisation strategy implemented	• Advanced automated testing, extensive coverage • Excellent CI/CD pipeline — automated releases • Full docker-compose setup, documented and working • Multi-stage builds, health checks, named volumes • Evidence of iteration and improvement
💡 <i>Section 4 is the most directly applicable criterion for today.</i> <i>A working docker-compose.yml + GitHub Actions CI = Merit. Add documentation, health checks, multi-stage builds = Distinction.</i>		

Getting Started — Right Now

Your action plan from today's session (and for the workshop)

1

Install Docker Desktop

docs.docker.com/get-started/get-docker/ (Windows / macOS / Linux)

2

Run the virtual robot

```
docker run -p 5000:5000 ghcr.io/francescodelduchetto/cmp9134_2526_robotsim:latest
```

3

Explore the API

Open <http://localhost:5000/docs> — your application will talk to this

4

Plan your Compose file

Which services do you need? Robot API + backend + frontend + database?

5

Add a devcontainer to your project

Create `.devcontainer/devcontainer.json` — or open in GitHub Codespaces

6

Add a GitHub Actions workflow

Build image → run tests → push to GHCR on every push to main

Resources

 Docker Desktop	Download & install	docs.docker.com/get-started/get-docker
 KillerCoda Docker	Browser-based interactive labs	killercoda.com/docker
 LabEx Docker Playground	PWD replacement, browser terminal	labex.io/tutorials/docker-online-docker-playground-372912
 GitHub Codespaces	Free for students (60 hr/month)	github.com/features/codespaces
 Docker for ROS2	Official Docker + robotics guide	docs.docker.com/guides/ros2/
 Docker Cheatsheet	Quick reference PDF	docs.docker.com/get-started/docker_cheatsheet.pdf
 containers.dev	DevContainer open standard	containers.dev
 Assessment Template	your friendly starting point	https://github.com/UoL-SoCS/CMP9134_2526_template

? Questions?

Then: Workshop — let's run some containers.

 mhanheide@lincoln.ac.uk

 lcas.lincoln.ac.uk

 JABAS.ai